

再探宝可梦、数码宝贝分类器

李宏毅《机器学习》2025 · 机器学习原理、泛化与模型复杂度



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 74 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=8>

目录

1 引言：为什么要重做宝可梦分类器？	3
2 Step 1：从图像到一个带未知参数的函数	4
2.1 观察：数码宝贝看起来更复杂	4
2.2 阈值模型：用一条线切开两类	6
3 Step 2：定义 Loss，衡量一个阈值好不好	7
4 理想与现实：我们想优化的对象拿不到	8
5 好训练集与坏训练集：抽样误差如何制造落差	10
6 我们真正想保证什么？	11
7 坏训练集的机率：Union Bound	13
8 Hoeffding Inequality：固定一个函数时会偏多远？	14
9 数字例子：上界有用，但常常很松	16
10 模型复杂度：从有限阈值到一般模型	17
11 复杂度的两难：现实接近理想，还是理想本身够好？	17
12 本讲综合：从玩具分类器看到机器学习原理	18

1 引言：为什么要重做宝可梦分类器？

这一讲表面上是在「再探」宝可梦与数码宝贝分类器，实际目标更大：借一个极简分类器说明机器学习里最核心的一件事，**我们在训练资料上看到的好表现，为什么不一定代表在真实世界上也好？**这也就是常说的 overfitting、generalization gap、训练 loss 与测试 loss 的落差。

上一讲相关例子里，模型曾因为 JPEG/JPG 档案格式的线索学到错误规则。P08 把图片格式统一后重新做一次，让问题回到机器学习本身：如果没有资料格式这种「捷径」，我们如何建立函数、定义损失、选择参数，又如何判断这个选择能否推广到没看过的资料？



Review: Basic Idea of ML



图 1: 机器学习的三步骤：先设定带未知参数的函数，再定义 loss，最后做 optimization。画面依据：约 01:00，字幕中老师复习 “function with unknown parameters / define loss / optimization”。

这三个步骤在前几讲已经反复出现。这里的重点不是再介绍一次流程，而是把流程放进一个可以完整分析的例子中：

- Step 1: 用一类函数 $\mathcal{H}$ 描述所有可能的分类规则。
- Step 2: 用 loss 衡量某个规则在某个资料集上错得多不多。
- Step 3: 从 $\mathcal{H}$ 里挑一个让训练 loss 最小的规则。

真正棘手的是第三步之后的问题：我们挑出来的是训练资料上最好的规则 h^{train} ，但我们真正希望它在所有未来可能遇到的资料上也好。训练资料只是全体资料的一小部分，二者之间一定存在抽样误差。于是本讲的问题可以写成：

训练资料上找到的好函数 $\implies$ 全体资料上也接近最好的函数？

Case Study: Pokémon v.s. Digimon



<https://medium.com/@tyreestevenson/teaching-a-computer-to-classify-anime-8c77bc89b881>

图 2: 案例任务: 输入一只宝可梦或数码宝贝, 输出它属于哪一类。画面依据: 约 02:30–04:30, 课堂用多组相似角色说明两类外观确实容易混淆。

本讲不是在追求最强分类器

这节课刻意使用非常简单的特征与阈值模型, 不是因为这样最先进, 而是因为它足够简单, 能把泛化理论讲清楚。若直接上深度网络, 模型、优化、资料、正则化、隐式偏好会同时纠缠在一起; 而阈值模型只有一个参数 h , 非常适合作为机器学习原理的显微镜。

2 Step 1: 从图像到一个带未知参数的函数

2.1 观察: 数码宝贝看起来更复杂

要做分类器, 第一步是想办法把输入 x 转成模型可以使用的特征。这里的 x 是一张角色图片。直接把整张图交给模型当然可以, 但那样很难做理论推导; 本讲选择一个人工设计的、可解释的特征: **线条复杂度**。

课堂里的直觉观察是: 宝可梦常常比较圆润、简洁, 数码宝贝常常有更多装甲、尖刺、机械结构, 边缘线条较密。这个观察未必对所有角色都成立, 但它可以成为一个可检验的假设: 如果数码宝贝的边缘像素数通常更多, 那我们可以用边缘数量做分类。

Observation

Digimon

線條較複雜？



Pokémon



線條較簡單？

图 3: 从视觉风格提出特征工程假设: 数码宝贝整体线条、装甲和细节较多, 宝可梦整体造型较简洁。画面依据: 约 06:15。

这个步骤很像传统机器学习里的 feature engineering。我们不是让模型从原始像素中自动学出表示, 而是先由人提出一个可能有用的统计量:

$e(x)$ = 图片 x 经过 edge detection 后的白色像素数.

如果 $e(x)$ 越大, 表示边缘越多, 图片越复杂; 如果 $e(x)$ 越小, 表示线条较少, 图片较简单。

Observation

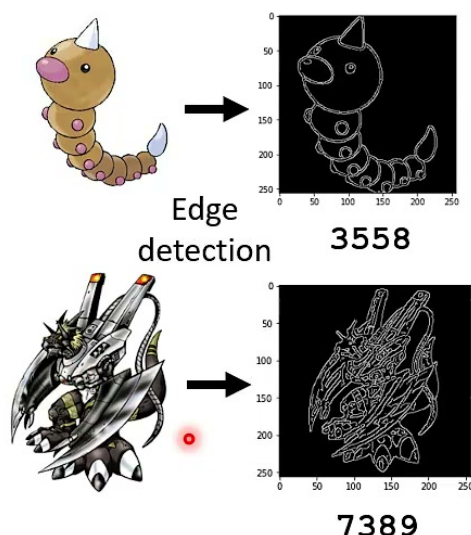


图 4: 边缘侦测后的白色像素数量可作为复杂度特征: 图中宝可梦例子为 3558, 数码宝贝例子为 7389。画面依据: 约 07:45–08:15。

特征工程在这里扮演什么角色?

原始图片是高维物件, 理论分析会很复杂。边缘像素数 $e(x)$ 把图片压缩成一个数字, 于是分类器只需要在一条数轴上切一刀。这个压缩一定会丢失资讯, 也一定会让某些样本分错; 但它让我们能清楚讨论「模型能力」「训练误差」「真实误差」之间的关系。

2.2 阈值模型: 用一条线切开两类

有了特征 $e(x)$, 最简单的分类规则就是设一个阈值 h :

$$f_h(x) = \begin{cases} \text{Digimon}, & e(x) \geq h, \\ \text{Pokemon}, & e(x) < h. \end{cases}$$

这里的未知参数就是 h 。不同的 h 对应不同的函数 f_h 。例如 h 很小的时候, 几乎所有图片都会被判成数码宝贝; h 很大的时候, 几乎所有图片都会被判成宝可梦。训练的目的就是从候选阈值里找到一个比较好的 h 。

Function with Unknown Parameters

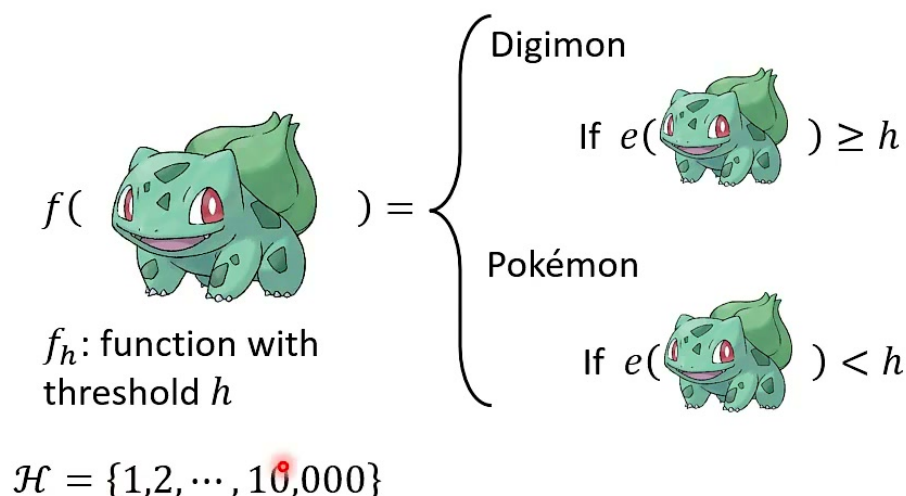


图 5: 阈值模型 f_h : 边缘数大于等于 h 判为 Digimon, 小于 h 判为 Pokemon; 候选函数集合设为 $\mathcal{H} = \{1, 2, \dots, 10000\}$ 。画面依据: 约 11:15–12:00。

课堂中把候选集合写成

$$\mathcal{H} = \{1, 2, \dots, 10000\}.$$

这代表我们只考虑一万个可能的阈值, 每个阈值对应一个函数。此时模型复杂度可以用 $|\mathcal{H}|$ 表示, 也就是「候选函数的数量」。

参数数量不等于这里的模型复杂度

这个模型只有一个参数 h , 但课堂里讨论的复杂度是 $|\mathcal{H}|$: 你允许模型从多少个函数里挑。若 h 只能取 1 到 100, 候选函数只有 100 个; 若能取 1 到 10000, 候选函数有 10000 个。参数维度相同, 候选空间大小却不同。深度学习里常说「参数多容易 overfit」, 背后的更一般说法是: 可选择的函数越多, 越容易在有限训练资料上找到一个偶然表现很好的函数。

3 Step 2: 定义 Loss, 衡量一个阈值好不好

给定资料集

$$\mathcal{D} = \{(x^1, \hat{y}^1), (x^2, \hat{y}^2), \dots, (x^N, \hat{y}^N)\},$$

其中 x^n 是第 n 张图片, $\hat{y}^n$ 是它的真实类别。对某个阈值 h , 先定义单笔资料上的 0/1 loss:

$$\ell(h, x^n, \hat{y}^n) = \mathbf{1}(f_h(x^n) \neq \hat{y}^n) = \begin{cases} 1, & f_h(x^n) \neq \hat{y}^n, \\ 0, & f_h(x^n) = \hat{y}^n. \end{cases}$$

它是在全体资料上最好的阈值，也就是这个模型家族 $\mathcal{H}$ 在真实世界中能做到的最好表现。

但现实中我们只有从 $\mathcal{D}_{all}$ 抽出来的一部分训练资料 $\mathcal{D}_{train}$ ，所以实际能做的是

$$h^{train} = \arg \min_{h \in \mathcal{H}} L(h, \mathcal{D}_{train}).$$

这就是训练过程：在训练集上找 loss 最小的 h 。问题是， h^{train} 是对 $\mathcal{D}_{train}$ 最优，不一定对 $\mathcal{D}_{all}$ 最优。



Training Examples

- If we can collect all Pokémons and Digimons in the universe $\mathcal{D}_{all}$, we can find the best threshold h^{all}

$$h^{all} = \arg \min_h L(h, \mathcal{D}_{all})$$

理想

- We only collect some examples $\mathcal{D}_{train}$ from $\mathcal{D}_{all}$

$$h^{train} = \arg \min_h L(h, \mathcal{D}_{train})$$

現實

图 7: 理想与现实的差别：若能拿到 $\mathcal{D}_{all}$ ，可直接求 h^{all} ；现实只能从抽样训练集 $\mathcal{D}_{train}$ 求 h^{train} 。画面依据：约 22:15–23:00。

课堂把 $\mathcal{D}_{train}$ 视为从 $\mathcal{D}_{all}$ 中 i.i.d. 抽样得到：

$$(x^1, \hat{y}^1), \dots, (x^N, \hat{y}^N) \stackrel{i.i.d.}{\sim} \mathcal{D}_{all}.$$

这里 i.i.d. 的意思是每笔资料独立、来自同一个分布。它是理论推导中非常关键的假设：若训练资料不是同分布，例如训练集只含某些年代、某些画风、某些档案来源的图片，那么训练 loss 就不能可靠代表真实 loss。

Testing data 在理论图像中的位置

真实的 $\mathcal{D}_{all}$ 通常不可得，所以实务上会留出 testing data 来估计 $L(h, \mathcal{D}_{all})$ 。但测试集只能用于评估，不能拿来反复调模型；一旦你根据测试集结果回头改 h 或改模型，测试集就变成训练过程的一部分，原本作为 $\mathcal{D}_{all}$ 代理的意义会被削弱。

5 好训练集与坏训练集：抽样误差如何制造落差

现在看两个抽样训练集。第一个训练集 $\mathcal{D}_{train,1}$ 的边缘数分布和 $\mathcal{D}_{all}$ 很接近。用它训练得到

$$h^{train1} = 4727, \quad L(h^{train1}, \mathcal{D}_{train,1}) = 0.27.$$

而全体资料上的理想阈值是

$$h^{all} = 4824, \quad L(h^{all}, \mathcal{D}_{all}) = 0.28.$$

两者非常接近，所以这是一个「好」训练集：它虽然只是抽样，但足够代表全体资料。

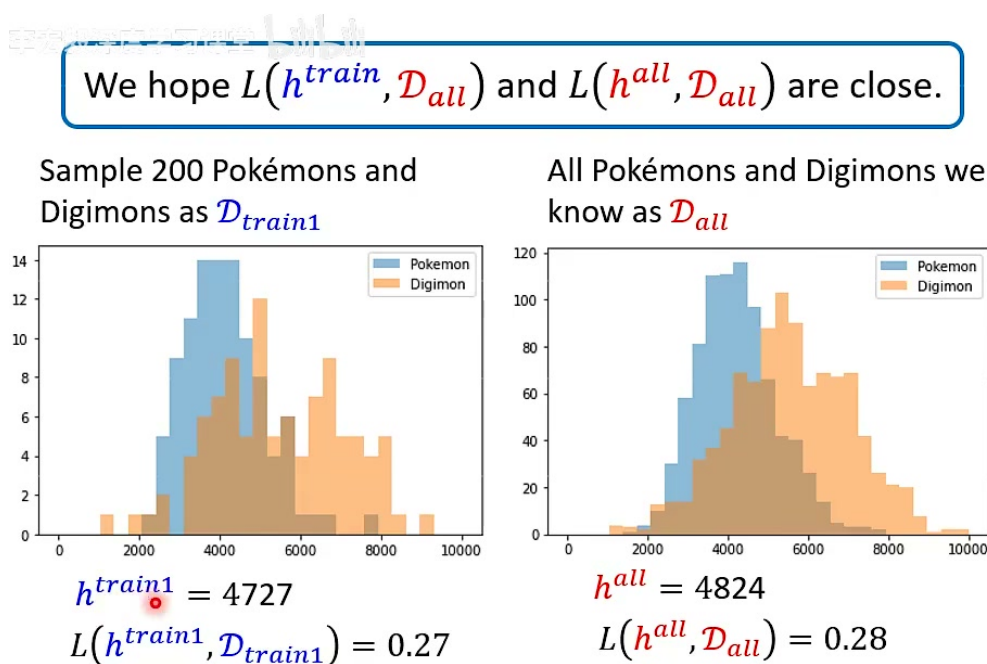


图 8: 好训练集例子: $\mathcal{D}_{train,1}$ 的分布接近 $\mathcal{D}_{all}$, 训练得到的 $h^{train1} = 4727$ 与理想 $h^{all} = 4824$ 接近, loss 也接近。画面依据: 约 27:45–28:30。

第二个训练集 $\mathcal{D}_{train,2}$ 则比较偏。它抽到的样本刚好让某个较低阈值看起来很好:

$$h^{train2} = 3642, \quad L(h^{train2}, \mathcal{D}_{train,2}) = 0.20.$$

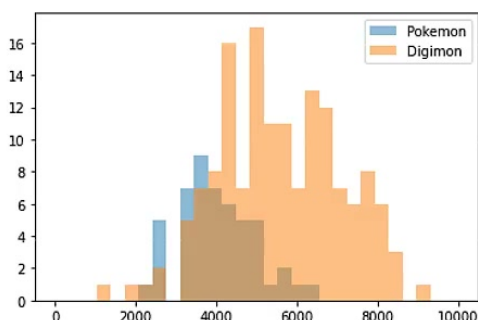
只看训练集, 0.20 甚至比前一个训练集的 0.27 更好。但把同一个阈值放回 $\mathcal{D}_{all}$ 上评估, 错误率变成

$$L(h^{train2}, \mathcal{D}_{all}) = 0.37.$$

也就是说, 训练集上看起来更漂亮的结果, 在真实全体资料上反而更差。

We hope $L(h^{train}, \mathcal{D}_{all})$ and $L(h^{all}, \mathcal{D}_{all})$ are close.

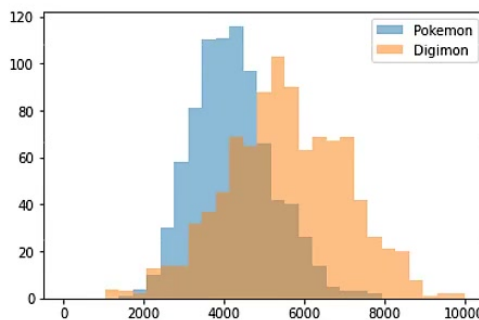
Sample 200 Pokémons and Digimons as $\mathcal{D}_{train2}$



$$h^{train2} = 3642$$

$$L(h^{train2}, \mathcal{D}_{train2}) = 0.20$$

All Pokémons and Digimons we know as $\mathcal{D}_{all}$



$$h^{all} = 4824$$

$$L(h^{all}, \mathcal{D}_{all}) = 0.28$$

$$L(h^{train2}, \mathcal{D}_{all}) = 0.37$$

图 9: 坏训练集例子: $\mathcal{D}_{train,2}$ 让 $h^{train2} = 3642$ 在训练集上只有 0.20 错误率, 但同一阈值在 $\mathcal{D}_{all}$ 上错误率为 0.37。画面依据: 约 32:15–32:35, 字幕明确提到“错误率是 37 percent”。

低 training loss 不等于低真实 loss

$\mathcal{D}_{train,2}$ 的例子就是 overfitting 的最小模型版本。模型不是学到荒谬规则, 也不是优化器坏掉; 它只是从有限训练集里挑到了一个「刚好适合这组样本」的阈值。训练资料越少、候选函数越多, 这种偶然性越容易发生。

这里要注意一个细节: 坏训练集并不是指「训练集中有错标」或「训练资料品质差」。即使每张图片标签都正确, 抽样也可能因为随机波动而偏离整体分布。坏训练集的意思是: 它让某些函数在训练集上的表现和在全体资料上的表现差太多。

6 我们真正想保证什么?

理想上, 我们希望现实中训练出来的 h^{train} , 放到 $\mathcal{D}_{all}$ 上时, 表现接近真正的全体最优 h^{all} :

$$L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all}) \leq \delta.$$

这里 δ 是一个小正数, 表示我们愿意接受的落差。如果 $\delta = 0.05$, 意思是 h^{train} 在真实世界上的错误率最多比理想阈值高 5 个百分点。

What do we want? $L(h^{train}, \mathcal{D}_{train})$ can be smaller than $L(h^{all}, \mathcal{D}_{all})$

We want $L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all}) \leq \delta$

What kind of $\mathcal{D}_{train}$ fulfil it?

$\forall h \in \mathcal{H}, |L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| \leq \delta/2$

图 10: 目标与充分条件: 希望 $L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all}) \leq \delta$; 若对所有 $h \in \mathcal{H}$ 都有训练 loss 与全体 loss 相差不超过 $\delta/2$, 则目标成立。画面依据: 约 36:15–37:30。

课堂给出的一个充分条件是:

$$\forall h \in \mathcal{H}, \quad |L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| \leq \frac{\delta}{2}.$$

这句话的含义很强: 不是只要求 h^{train} 一个函数估得准, 也不是只要求 h^{all} 一个函数估得准, 而是要求候选集合里的每个函数, 在训练集和全体资料上的 loss 都接近。

为什么这个条件足够? 推导可以分成三步:

$$\begin{aligned} L(h^{train}, \mathcal{D}_{all}) &\leq L(h^{train}, \mathcal{D}_{train}) + \frac{\delta}{2} && \text{因为 } h^{train} \text{ 的训练/全体 loss 接近} \\ &\leq L(h^{all}, \mathcal{D}_{train}) + \frac{\delta}{2} && \text{因为 } h^{train} \text{ 是训练集上最小 loss 的选择} \\ &\leq L(h^{all}, \mathcal{D}_{all}) + \delta && \text{因为 } h^{all} \text{ 的训练/全体 loss 也接近.} \end{aligned}$$

把最后一行移项, 就得到

$$L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all}) \leq \delta.$$

为什么要对所有 h 都成立?

因为 h^{train} 本身是看过 $\mathcal{D}_{train}$ 后才选出来的。你不能先固定一个 h , 证明它估得准, 然后再根据训练集挑另一个 h 。为了允许训练过程在 $\mathcal{H}$ 中自由选择, 必须确保每个候选函数的训练 loss 都不会严重偏离全体 loss。这就是 uniform convergence 的直觉版本。

于是,「好训练集」可以被正式定义为: 对所有候选函数 $h \in \mathcal{H}$, 训练集 loss 都能可靠近似全体 loss。反过来, 坏训练集就是至少存在一个 h , 使得训练 loss 与全体 loss 差太多。

7 坏训练集的机率：Union Bound

接下来要问：抽到坏训练集的机率有多大？

先把坏训练集写成事件。给定误差容忍度 ϵ ，如果某个训练集满足

$$\exists h \in \mathcal{H}, \quad |L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| > \epsilon,$$

就称它是坏训练集。也就是说，只要有一个候选函数 h 被训练集「误导」得太严重，这组训练集就坏了。

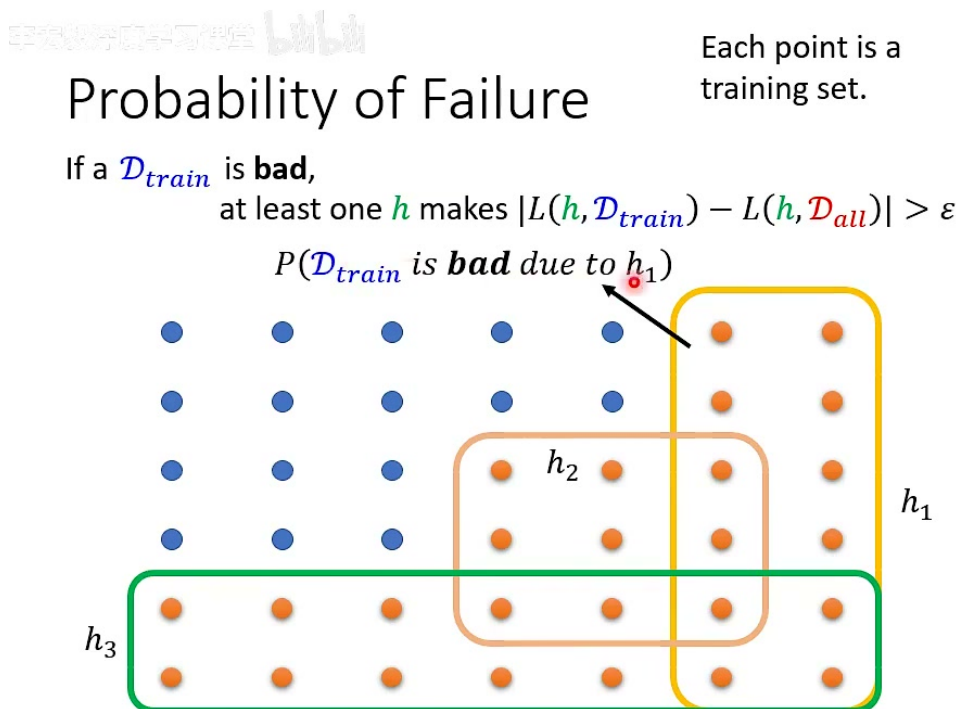


图 11: 坏训练集的集合图像：每个点是一组可能抽到的训练集；橘色点代表坏训练集。若某组训练集坏掉，至少有一个 h 让 $|L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| > \epsilon$ 。画面依据：约 48:15–49:30。

对每个固定的 h ，定义事件

$$A_h = \{\mathcal{D}_{train} : |L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| > \epsilon\}.$$

A_h 表示「训练集因为这个 h 而坏」。整体坏事件就是这些事件的并集：

$$\{\mathcal{D}_{train} \text{ is bad}\} = \bigcup_{h \in \mathcal{H}} A_h.$$

并集的机率可以用 union bound 上界：

$$P(\mathcal{D}_{train} \text{ is bad}) = P\left(\bigcup_{h \in \mathcal{H}} A_h\right) \leq \sum_{h \in \mathcal{H}} P(A_h).$$

Union bound 为什么只是上界？

不同 A_h 之间可能重叠：同一组训练集可能同时被多个 h 弄坏。把每个 $P(A_h)$ 直接加起来，会把重叠部分重复计算，所以得到的是 upper bound，不一定紧。这个「不紧」很重要，因为后面会看到上界甚至可能大于 1；那不是机率真的大于 1，而是这个粗上界暂时没有提供有用信息。

这一步已经出现模型复杂度的影子：如果 $\mathcal{H}$ 里有更多函数，就有更多事件 A_h 可以导致训练集坏掉。即使每个单独的 h 出问题机率都很小，候选函数很多时，总体出问题机率也会被放大。

8 Hoeffding Inequality: 固定一个函数时会偏多远？

Union bound 把问题化成估计每个 $P(A_h)$ 。现在固定某个 h ，看

$$L(h, \mathcal{D}_{train}) = \frac{1}{N} \sum_{n=1}^N \ell(h, x^n, \hat{y}^n).$$

在 h 固定时，每一项 $\ell(h, x^n, \hat{y}^n)$ 都是一个介于 0 和 1 之间的随机变量。若样本 i.i.d. 来自 $\mathcal{D}_{all}$ ，这些随机变量也是独立同分布的。它们的平均值 $L(h, \mathcal{D}_{train})$ 是经验平均；它们的期望就是全体资料上的错误率 $L(h, \mathcal{D}_{all})$ 。

Hoeffding inequality 告诉我们，对固定 h ：

$$P(|L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| > \epsilon) \leq 2 \exp(-2N\epsilon^2).$$

也就是

$$P(A_h) \leq 2 \exp(-2N\epsilon^2).$$

把它代回 union bound：

$$\begin{aligned} P(\mathcal{D}_{train} \text{ is bad}) &\leq \sum_{h \in \mathcal{H}} P(A_h) \\ &\leq \sum_{h \in \mathcal{H}} 2 \exp(-2N\epsilon^2) \\ &= |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2). \end{aligned}$$

$$\begin{aligned}
 P(\mathcal{D}_{train} \text{ is bad}) &= \bigcup_{h \in \mathcal{H}} P(\mathcal{D}_{train} \text{ is bad due to } h) \\
 &\leq \sum_{h \in \mathcal{H}} P(\mathcal{D}_{train} \text{ is bad due to } h) \\
 &\leq \sum_{h \in \mathcal{H}} 2 \exp(-2N\epsilon^2) \\
 &= |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2)
 \end{aligned}$$

How to make $P(\mathcal{D}_{train} \text{ is bad})$ smaller?

Larger N and smaller $|\mathcal{H}|$

图 12: 坏训练集机率上界: $P(\mathcal{D}_{train} \text{ is bad}) \leq |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2)$ 。画面依据: 约 57:45–58:30。

这个公式读出来是什么？

$$P(\mathcal{D}_{train} \text{ is bad}) \leq |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2).$$

右边有两个方向相反的力量：

- N 越大, $\exp(-2N\epsilon^2)$ 越小, 坏训练集机率指数下降。
- $|\mathcal{H}|$ 越大, 候选函数越多, 上界线性变大。

这就是「资料越多越不容易 overfit；模型越复杂越容易 overfit」的一种理论版本。

如果希望坏训练集机率最多为 δ , 只要让

$$|\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2) \leq \delta.$$

解这个不等式：

$$\begin{aligned}
 2|\mathcal{H}| \exp(-2N\epsilon^2) &\leq \delta, \\
 \exp(-2N\epsilon^2) &\leq \frac{\delta}{2|\mathcal{H}|}, \\
 -2N\epsilon^2 &\leq \log \frac{\delta}{2|\mathcal{H}|}, \\
 N &\geq \frac{\log(2|\mathcal{H}|/\delta)}{2\epsilon^2}.
 \end{aligned}$$

这说明样本数只需要随 $\log |\mathcal{H}|$ 增长, 而不是随 $|\mathcal{H}|$ 本身线性增长; 但它也会随 $1/\epsilon^2$ 增长。想把训练/真实误差估得非常精确, 样本需求会快速变大。

9 数字例子：上界有用，但常常很松

课堂使用

$$|\mathcal{H}| = 10000, \quad \epsilon = 0.1$$

做三个例子。把它代入

$$P(\mathcal{D}_{train} \text{ is bad}) \leq |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2),$$

得到：

候选函数数 $ \mathcal{H} $	样本数 N	容忍误差 ϵ	坏训练集机率上界
10000	100	0.1	≤ 2707
10000	500	0.1	≤ 0.91
10000	1000	0.1	≤ 0.00004

手书板应学写书 Bilibili

Example

$$\begin{aligned} \mathcal{H} &= \{1, 2, \dots, 10,000\} \\ \mathcal{D}_{train} &= \{(x^1, \hat{y}^1), (x^2, \hat{y}^2), \dots, (x^N, \hat{y}^N)\} \\ \forall h \in \mathcal{H}, |L(h, \mathcal{D}_{train}) - L(h, \mathcal{D}_{all})| &\leq \epsilon \end{aligned}$$

$$P(\mathcal{D}_{train} \text{ is bad}) \leq |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2)$$

$$|\mathcal{H}| = 10000, N = 100, \epsilon = 0.1 \quad \text{Usually happen QQ}$$

$$P(\mathcal{D}_{train} \text{ is bad}) \leq 2707$$

$$|\mathcal{H}| = 10000, N = 500, \epsilon = 0.1$$

$$P(\mathcal{D}_{train} \text{ is bad}) \leq 0.91$$

$$|\mathcal{H}| = 10000, N = 1000, \epsilon = 0.1$$

$$P(\mathcal{D}_{train} \text{ is bad}) \leq 0.00004$$

图 13: 数值例子：当 $|\mathcal{H}| = 10000, \epsilon = 0.1$ 时， $N = 100$ 的上界大于 1； $N = 500$ 时降到 0.91； $N = 1000$ 时降到 0.00004。画面依据：约 62:15–62:30。

第一个上界 2707 看起来很奇怪，因为机率不可能大于 1。正确理解是：这个 bound 太松了，在 $N = 100$ 时没有提供有效保证。它并不是说坏训练集机率真的等于 2707，而是说我们目前的粗估方法只能给出一个没用的上界。

第二个例子 0.91 仍然很松，但已经进入机率范围。第三个例子 0.00004 则说明样本数增加的效果非常强，因为指数项会随着 N 增加快速下降。

不要把 bound 当成真实机率

泛化上界通常回答的是「最坏情况下能保证什么」，不是「实际实验中会发生什么」。它可能非常保守，却仍然有解释价值：它清楚指出影响坏训练集机率的因素是样本数 N 、容忍误差 ϵ 与函数集合大小 $|\mathcal{H}|$ 。

10 模型复杂度：从有限阈值到一般模型

在这个阈值例子中，模型复杂度就是 $|\mathcal{H}|$ ：我们允许 h 有多少个候选值。若 $\mathcal{H} = \{1, \dots, 10000\}$ ，复杂度就是 10000。更大的 $\mathcal{H}$ 代表模型有更多选择，也就更容易在训练集上找到偶然好的函数。

这也解释了前面常听到的一句话：「参数越多，越容易 overfitting。」严格说，参数数量只是复杂度的一种粗略指标。真正影响泛化的是函数集合的容量：模型能表示多少不同规则、能在资料上切出多复杂的分类边界。

课堂里进一步提到：如果参数是连续值， $|\mathcal{H}|$ 看起来会变成无限大。直观回答是，现实计算机的浮点数不是无限精度，所以实际可表示的函数仍然是有限的；更正式的理论则会使用 VC dimension 等概念衡量模型容量。本讲不展开 VC dimension，但它和这里的 $|\mathcal{H}|$ 扮演类似角色：描述模型家族到底有多能「配合」资料。

从 $|\mathcal{H}|$ 到 VC dimension 的直觉

有限集合时，复杂度可以直接数函数个数。连续参数模型不能简单数个数，于是理论会问另一种问题：这个模型最多能把多少点用所有可能方式分开？能分开的点越多，模型容量越大。VC dimension 就是这类想法的经典形式。

11 复杂度的两难：现实接近理想，还是理想本身够好？

到目前为止，我们得到一个看似简单的建议：让 N 大一点，让 $|\mathcal{H}|$ 小一点。样本数大当然好，但 $|\mathcal{H}|$ 小并不总是好。因为 $|\mathcal{H}|$ 变小以后，理想函数 h^{all} 也只能在更小的集合里挑，可能导致

$$L(h^{all}, \mathcal{D}_{all})$$

本身变大。

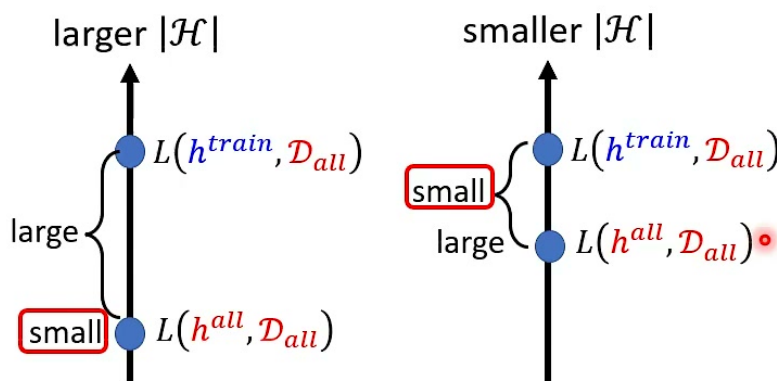
这就是模型复杂度的两难：

- 若 $|\mathcal{H}|$ 很大，理想模型可能很好，因为候选函数多，总能找到更贴合真实分布的规则；但训练资料有限时， h^{train} 可能和 h^{all} 差很远。
- 若 $|\mathcal{H}|$ 很小， h^{train} 比较容易接近 h^{all} ；但 h^{all} 本身可能很差，因为模型家族太弱，连理想状态下也无法表达正确规则。

Tradeoff of Model Complexity

Larger N and smaller $|\mathcal{H}| \Rightarrow L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all}) \leq \delta$

Smaller $|\mathcal{H}| \Rightarrow$ Larger $L(h^{all}, \mathcal{D}_{all})$



魚與熊掌可以兼得嗎？ Yes, Deep Learning.

图 14: 模型复杂度的 tradeoff: 小 $|\mathcal{H}|$ 让现实接近理想, 但可能让理想本身变差; 大 $|\mathcal{H}|$ 让理想更好, 却可能扩大现实与理想的距离。画面依据: 约 73:45–74:00。

课堂最后用「鱼与熊掌可以兼得吗?」引出深度学习。这里的鱼与熊掌分别是:

理想要好 以及 现实要接近理想。

传统泛化上界看起来告诉我们: 模型越大, 越容易坏; 但深度学习实践中, 巨大模型在足够资料、合适优化、正则化、结构偏好与训练技巧下, 往往可以同时拥有强表达力和良好泛化。这是后续课程要继续解释的问题。

这讲的核心张力

$$L(h^{train}, \mathcal{D}_{all}) = L(h^{all}, \mathcal{D}_{all}) + [L(h^{train}, \mathcal{D}_{all}) - L(h^{all}, \mathcal{D}_{all})].$$

第一项是模型家族的表达能力问题: 理想最好能有多好。第二项是泛化问题: 训练出来的现实离理想有多远。模型太小, 第一项大; 模型太大且资料不足, 第二项可能大。机器学习很多设计都在平衡这两项。

12 本讲综合: 从玩具分类器看到机器学习原理

这节课用一个非常朴素的阈值分类器, 串起了机器学习原理中几个关键概念:

- 模型家族: $\mathcal{H} = \{1, \dots, 10000\}$, 每个阈值对应一个分类函数。
- 训练目标: 在训练集 $\mathcal{D}_{train}$ 上找 $h^{train} = \arg \min_h L(h, \mathcal{D}_{train})$ 。

- **理想目标**：真正想要的是接近 $h^{all} = \arg \min_h L(h, \mathcal{D}_{all})$ 。
- **泛化条件**：若所有 h 的训练 loss 都接近全体 loss，则 h^{train} 的真实表现接近 h^{all} 。
- **坏训练集机率**：由 union bound 和 Hoeffding inequality 得到

$$P(\mathcal{D}_{train} \text{ is bad}) \leq |\mathcal{H}| \cdot 2 \exp(-2N\epsilon^2).$$

- **复杂度权衡**：小模型较稳但可能能力不足；大模型能力强但需要更多资料和更好的训练机制来避免现实偏离理想。

一句话总结

训练并不是在「真世界」上直接找最佳函数，而是在有限样本上找看起来最佳的函数。泛化理论要回答的是：有限样本上的最佳，什么时候能接近真世界里的最佳？这节课给出的答案是：训练集要对所有候选函数都足够代表真实分布；而这种代表性会随样本数增加而变好，随模型复杂度增加而更难保证。

把这堂课和前几讲连起来看，机器学习不是单纯把 loss 降低就结束。优化负责让训练 loss 下降，泛化负责解释训练 loss 是否可信，模型复杂度则同时影响理想能力和泛化风险。后续进入深度学习时，真正的问题会变成：为什么一个极大的函数集合没有像这个简单 bound 暗示的那样必然崩坏？这正是现代深度学习理论与实务最有趣的地方之一。